

all oops basic to advanced concept in very easy word and types and its explanation def and code dry run

Object-Oriented Programming (OOP) - Complete Guide

What is OOP?

OOP is a programming style where we organize code around **objects** (things) rather than functions and logic. Think of it like building with Lego blocks - each block is an object that can be used and reused.

4 MAIN PILLARS OF OOP

1. ENCAPSULATION (Data Hiding)

Easy meaning: Wrapping data and methods together, hiding internal details.

Real-life example: A TV remote - you press buttons (methods) without knowing the internal circuits (data).

```
python

# WITHOUT Encapsulation (Bad)
class BankAccount:
    def __init__(self, balance):
        self.balance = balance # Directly accessible - DANGEROUS!

# Anyone can change balance directly
acc = BankAccount(1000)
acc.balance = -500 # BAD! Can set negative balance

# WITH Encapsulation (Good)
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # Private (hidden)

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
```

```

def withdraw(self, amount):
    if 0 < amount <= self.__balance:
        self.__balance -= amount

def get_balance(self):
    return self.__balance

# Dry Run:
acc = BankAccount(1000)
print(acc.get_balance()) # 1000
acc.deposit(500)          # balance becomes 1500
acc.withdraw(200)         # balance becomes 1300
# acc.__balance - ERROR! Can't access directly

```

2. INHERITANCE (Reusing Code)

Easy meaning: Child class gets all features from parent class.

Real-life example: You inherit traits from your parents.

python

```

class Vehicle: # Parent class
    def __init__(self, brand, wheels):
        self.brand = brand
        self.wheels = wheels

    def start(self):
        return "Vehicle started"

class Car(Vehicle): # Child class (inherits from Vehicle)
    def __init__(self, brand, model):
        super().__init__(brand, 4) # Call parent
        self.model = model

    def honk(self):
        return "Beep beep!"

class Bike(Vehicle): # Another child
    def __init__(self, brand):
        super().__init__(brand, 2)

# Dry Run:
car = Car("Toyota", "Camry")
print(car.brand) # Toyota (inherited)

```

```

print(car.wheels)    # 4 (inherited)
print(car.start())   # Vehicle started (inherited)
print(car.honk())    # Beep beep! (own method)

bike = Bike("Honda")
print(bike.wheels)   # 2 (inherited)

```

Types of Inheritance:

1. **Single:** One parent, one child
2. **Multiple:** Child inherits from multiple parents
3. **Multilevel:** Grandparent → Parent → Child
4. **Hierarchical:** One parent, multiple children

3. POLYMORPHISM (Many Forms)

Easy meaning: Same method name, different behavior.

Real-life example: "Play" - play guitar, play football, play video game (different actions).

```

python

# Example 1: Method Overriding
class Animal:
    def speak(self):
        return "Animal makes sound"

class Dog(Animal):
    def speak(self): # Overriding parent method
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

# Example 2: Duck Typing
class Duck:
    def make_sound(self):
        return "Quack"

class Phone:
    def make_sound(self):
        return "Ring ring"

```

```

def sound_maker(obj):
    print(obj.make_sound()) # Works for any object with make_sound()

# Dry Run:
animals = [Dog(), Cat(), Animal()]
for animal in animals:
    print(animal.speak())
# Output: Woof!, Meow!, Animal makes sound

sound_maker(Duck()) # Quack
sound_maker(Phone()) # Ring ring

```

4. ABSTRACTION (Hiding Complexity)

Easy meaning: Show only essential features, hide background details.

Real-life example: Car steering wheel - you turn it without knowing engine mechanics.

```

python

from abc import ABC, abstractmethod

class Shape(ABC): # Abstract class
    @abstractmethod
    def area(self): # Abstract method (must be implemented)
        pass

    @abstractmethod
    def perimeter(self):
        pass

    def display(self): # Regular method (optional)
        print("This is a shape")

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self): # Must implement
        return self.length * self.width

    def perimeter(self): # Must implement
        return 2 * (self.length + self.width)

```



```

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius * self.radius

    def perimeter(self):
        return 2 * 3.14 * self.radius

# Dry Run:
rect = Rectangle(5, 3)
print(rect.area())      # 15
print(rect.perimeter()) # 16
rect.display()          # This is a shape

# shape = Shape() # ERROR! Can't create abstract class object

```

OTHER IMPORTANT OOP CONCEPTS

CLASS vs OBJECT

```

python

# Class = Blueprint
class Car:
    def __init__(self, color):
        self.color = color

# Object = Actual thing built from blueprint
car1 = Car("Red")
car2 = Car("Blue")
# car1 and car2 are different objects

```

CONSTRUCTOR (__init__)

```

python

class Student:
    # Constructor - runs when object is created

```

```
def __init__(self, name, age):
    self.name = name    # Instance variable
    self.age = age
    print(f"Student {name} created!")

# When object created, constructor auto-runs
s1 = Student("Alice", 20) # Output: Student Alice created!
```

INSTANCE vs CLASS VARIABLES

python

```
class Employee:
    company = "Google" # Class variable (shared by all)

    def __init__(self, name):
        self.name = name # Instance variable (unique for each)

e1 = Employee("John")
e2 = Employee("Jane")

print(e1.company) # Google
print(e2.company) # Google
print(e1.name)    # John
print(e2.name)    # Jane

# Change class variable affects all
Employee.company = "Microsoft"
print(e1.company) # Microsoft
print(e2.company) # Microsoft
```

STATIC METHODS

python

```
class MathOperations:
    @staticmethod # Doesn't need self
    def add(x, y):
        return x + y

    @staticmethod
    def multiply(x, y):
```

```

        return x * y

# Call without creating object
print(MathOperations.add(5, 3))      # 8
print(MathOperations.multiply(5, 3)) # 15

```

CLASS METHODS

python

```

class Person:
    count = 0

    def __init__(self, name):
        self.name = name
        Person.count += 1

    @classmethod
    def get_count(cls): # cls = class
        return cls.count

p1 = Person("Alice")
p2 = Person("Bob")
print(Person.get_count()) # 2

```

METHOD OVERLOADING (Python doesn't support directly)

python

```

# Python alternative - use default arguments
class Calculator:
    def add(self, a, b, c=0):
        return a + b + c

calc = Calculator()
print(calc.add(5, 3))      # 8 (uses c=0)
print(calc.add(5, 3, 2))   # 10 (uses c=2)

```

OPERATOR OVERLOADING

python

```

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other): # Overload + operator
        return Point(self.x + other.x, self.y + other.y)

    def __str__(self): # Overload print()
        return f"({self.x}, {self.y})"

p1 = Point(2, 3)
p2 = Point(4, 5)
p3 = p1 + p2 # Uses __add__
print(p3)    # (6, 8)

```

MAGIC METHODS (Dunder Methods)

python

```

class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __len__(self): # For len()
        return self.pages

    def __str__(self): # For print()
        return f"Book: {self.title}"

    def __eq__(self, other): # For ==
        return self.pages == other.pages

book1 = Book("Python", 300)
book2 = Book("Java", 300)
print(len(book1)) # 300
print(book1)      # Book: Python
print(book1 == book2) # True

```

MULTIPLE INHERITANCE

python

```
class Flyer:
    def fly(self):
        return "Flying"

class Swimmer:
    def swim(self):
        return "Swimming"

class Duck(Flyer, Swimmer): # Multiple inheritance
    pass

duck = Duck()
print(duck.fly()) # Flying
print(duck.swim()) # Swimming
```

DIAMOND PROBLEM (Multiple Inheritance Issue)

python

```
class A:
    def method(self):
        print("A")

class B(A):
    def method(self):
        print("B")

class C(A):
    def method(self):
        print("C")

class D(B, C): # Which method? B or C?
    pass

d = D()
d.method() # B (Python uses MRO - Method Resolution Order)
print(D.__mro__) # Shows order: D -> B -> C -> A
```

COMPOSITION (Has-A Relationship)

python

```
class Engine:
    def start(self):
        return "Engine started"

class Wheels:
    def rotate(self):
        return "Wheels rotating"

class Car:
    def __init__(self):
        self.engine = Engine() # Has-A relationship
        self.wheels = Wheels()

    def drive(self):
        return f"{self.engine.start()} and {self.wheels.rotate()}"

car = Car()
print(car.drive()) # Engine started and Wheels rotating
```

PROPERTY DECORATOR (Getters/Setters)

python

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def celsius(self): # Getter
        return self._celsius

    @celsius.setter
    def celsius(self, value): # Setter
        if value < -273.15:
            raise ValueError("Below absolute zero!")
        self._celsius = value

    @property
    def fahrenheit(self): # Computed property
        return (self._celsius * 9/5) + 32
```

```
temp = Temperature(25)
print(temp.celsius)      # 25 (uses getter)
temp.celsius = 30        # 30 (uses setter)
print(temp.fahrenheit)   # 86.0
# temp.celsius = -300     # ERROR! Validation fails
```

PRIVATE vs PROTECTED vs PUBLIC

python

```
class Example:
    public = "Anyone can access"          # Public
    _protected = "Accessible in subclass" # Protected (convention)
    __private = "Only inside class"       # Private (name mangling)

    def show(self):
        print(self.__private) # Works inside class

# _protected - convention, but still accessible
# __private - name mangled to _Example__private
```

PRACTICAL PROJECT EXAMPLE

python

```
# Complete Library Management System
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self._is_borrowed = False # Encapsulation

    def borrow(self):
        if not self._is_borrowed:
            self._is_borrowed = True
            return f'{self.title} borrowed'
        return f'{self.title} unavailable'

    def return_book(self):
        if self._is_borrowed:
            self._is_borrowed = False
```

```

        return f"'{self.title}' returned"
        return f"'{self.title}' wasn't borrowed"

def __str__(self):
    return f"{self.title} by {self.author}"

class Library:
    def __init__(self, name):
        self.name = name
        self.books = []

    def add_book(self, book):
        self.books.append(book)
        return f"Added: {book.title}"

    def find_book(self, title):
        for book in self.books:
            if book.title.lower() == title.lower():
                return book
        return None

    def show_books(self):
        if not self.books:
            return "No books available"
        return "\n".join(str(book) for book in self.books)

# Usage
lib = Library("City Library")
book1 = Book("1984", "George Orwell", "123")
book2 = Book("Python", "Guido", "456")

lib.add_book(book1)
lib.add_book(book2)

print(book1.borrow()) # '1984' borrowed
print(book1.borrow()) # '1984' unavailable
print(book1.return_book()) # '1984' returned

```

SUMMARY TABLE

Concept	Easy Definition	Keywords
Encapsulation	Hide data, expose methods	<code>__private</code> , getters/setters

Concept	Easy Definition	Keywords
Inheritance	Child gets parent features	<code>class Child(Parent)</code>
Polymorphism	Same name, different behavior	Method overriding, duck typing
Abstraction	Show only essentials	<code>@abstractmethod</code> , ABC
Class	Blueprint	<code>class Name:</code>
Object	Instance of class	<code>obj = Class()</code>
Constructor	Runs when object created	<code>__init__</code>
Static Method	No self/class needed	<code>@staticmethod</code>
Class Method	Works with class	<code>@classmethod</code>
Magic Method	Special double underscore	<code>__add__</code> , <code>__str__</code>

QUICK TIPS

1. **Always use** `self` for instance methods
2. **Use** `__init__` to initialize objects
3. **Use inheritance** for "IS-A" relationship
4. **Use composition** for "HAS-A" relationship
5. **Keep data private** using `__` (encapsulation)
6. **Override methods** for polymorphism
7. **Use abstract classes** when objects should not be created directly

basics to advanced oops

COMPLETE OOP GUIDE: Basics to Advanced 🚀

TABLE OF CONTENTS

1. OOP Basics
2. Classes & Objects

3. 4 Pillars of OOP
4. Advanced Concepts
5. Design Patterns
6. Best Practices
7. Real-World Examples

BASICS {#basics}

What is OOP?

OOP = **Programming with Objects**. Think of it like cooking with recipes:

- **Class** = Recipe card (blueprint)
- **Object** = Actual dish you cook (instance)
- **Method** = Steps in recipe (functions)
- **Attribute** = Ingredients (variables)

Why OOP?

- ✓ **Reusability** - Write once, use many times
- ✓ **Modularity** - Break big problems into small pieces
- ✓ **Security** - Hide sensitive data
- ✓ **Easy maintenance** - Fix one part without breaking others

CLASSES & OBJECTS {#classes-objects}

1. Basic Class Structure

```
python

# CLASS = Blueprint
class Car:
    # Class variable (shared by all objects)
    wheels = 4

    # Constructor - runs when object is created
```

```

def __init__(self, brand, color):
    # Instance variables (unique to each object)
    self.brand = brand
    self.color = color
    self.speed = 0 # Default value

# Method (function inside class)
def accelerate(self):
    self.speed += 10
    return f"Speed: {self.speed} km/h"

def brake(self):
    self.speed -= 5
    return f"Speed: {self.speed} km/h"

def display(self):
    return f"{self.color} {self.brand}"

# OBJECT = Instance of class
car1 = Car("Toyota", "Red") # Creating object
car2 = Car("Honda", "Blue")

# Dry Run
print(car1.display())      # Red Toyota
print(car1.accelerate())   # Speed: 10 km/h
print(car1.accelerate())   # Speed: 20 km/h
print(car1.brake())        # Speed: 15 km/h
print(car1.wheels)         # 4 (class variable)
print(car2.wheels)         # 4 (shared)

```

2. self Keyword Explained

python

```

class Person:
    def __init__(self, name):
        self.name = name # "self" = current object

    def greet(self):
        return f"Hi, I'm {self.name}"

# self refers to the object calling the method
p1 = Person("Alice")
p2 = Person("Bob")

```

```
print(p1.greet()) # Hi, I'm Alice (self = p1)
print(p2.greet()) # Hi, I'm Bob   (self = p2)
```

3. Constructor Types

python

```
# 1. Default Constructor (no parameters)
class DefaultDemo:
    def __init__(self):
        self.value = 0

# 2. Parameterized Constructor
class Student:
    def __init__(self, name, age, grade="A"): # Default parameter
        self.name = name
        self.age = age
        self.grade = grade

# 3. Copy Constructor (manual)
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def copy(self):
        return Book(self.title, self.author)

# Usage
s1 = Student("John", 20)          # grade = "A"
s2 = Student("Jane", 22, "B")    # grade = "B"
book1 = Book("Python", "Guido")
book2 = book1.copy()             # Creates new object
```

4 PILLARS OF OOP {#pillars}

PILLAR 1: ENCAPSULATION (Data Hiding)

Definition: Bundle data and methods together, hide internal details.

Without Encapsulation (BAD)

python

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance # Public - anyone can change

# Problem: Anyone can modify balance directly
acc = BankAccount(1000)
acc.balance = -500 # BUG! Negative balance allowed
acc.balance = 999999 # BUG! No validation
```

With Encapsulation (GOOD)

python

```
class BankAccount:
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.__balance = balance # Private (__ makes it hidden)
        self.__transactions = [] # Private List

# Getter method
def get_balance(self):
    return self.__balance

# Setter method with validation
def deposit(self, amount):
    if amount > 0:
        self.__balance += amount
        self.__transactions.append(f"+{amount}")
        return f"Deposited ${amount}"
    return "Invalid amount"

def withdraw(self, amount):
    if 0 < amount <= self.__balance:
        self.__balance -= amount
        self.__transactions.append(f"-{amount}")
        return f"Withdrew ${amount}"
    return "Insufficient balance"

def get_transactions(self):
    return self.__transactions.copy() # Return copy for safety

def __calculate_interest(self): # Private method
    return self.__balance * 0.05
```

```
# Usage
acc = BankAccount("Alice", 1000)
print(acc.get_balance())      # 1000
acc.deposit(500)              # Deposited $500
acc.withdraw(200)             # Withdrew $200
print(acc.get_balance())      # 1300
# print(acc.__balance)        # ERROR! Private
# print(acc.__transactions)   # ERROR! Private
```

Types of Access Modifiers

python

```
class AccessDemo:
    public = "Everyone can see me"      # Public
    _protected = "Subclasses can see me" # Protected (convention)
    __private = "Only I can see me"     # Private (name mangling)

    def __init__(self):
        self.__private_var = "Secret"

    def __private_method(self):
        return "Can't call from outside"

    def public_method(self):
        return self.__private_method() # OK - called inside class

# Name mangling: __private becomes _ClassName__private
obj = AccessDemo()
print(obj.public)      # Everyone can see me
print(obj._protected)  # Subclasses can see me (just convention)
# print(obj.__private) # ERROR
print(obj._AccessDemo__private) # Works but DON'T DO THIS!
```

PILLAR 2: INHERITANCE (Code Reuse)

Definition: Child class gets all features from parent class.

Basic Inheritance

python

```

# Parent/Super/Base class
class Animal:
    def __init__(self, name, species):
        self.name = name
        self.species = species

    def eat(self):
        return f"{self.name} is eating"

    def sleep(self):
        return f"{self.name} is sleeping"

# Child/Sub/Derived class
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name, "Canine") # Call parent constructor
        self.breed = breed

    # Override parent method
    def eat(self):
        return f"{self.name} the dog is eating dog food"

    # New method specific to Dog
    def bark(self):
        return f"{self.name} says Woof!"

class Cat(Animal):
    def __init__(self, name):
        super().__init__(name, "Feline")

    def meow(self):
        return f"{self.name} says Meow!"

# Dry Run
dog = Dog("Buddy", "Golden Retriever")
cat = Cat("Whiskers")

print(dog.eat())      # Buddy the dog is eating dog food (overridden)
print(dog.sleep())    # Buddy is sleeping (inherited)
print(dog.bark())     # Buddy says Woof! (own method)
print(cat.meow())     # Whiskers says Meow!

```

Types of Inheritance

1. Single Inheritance (One parent → One child)

```
python
```

```
class Parent:
    pass

class Child(Parent):
    pass
```

2. Multiple Inheritance (Multiple parents → One child)

```
python
```

```
class Flyer:
    def move(self):
        return "Flying"

class Swimmer:
    def move(self):
        return "Swimming"

class Duck(Flyer, Swimmer): # Inherits from both
    pass

duck = Duck()
print(duck.move()) # Flying (first parent wins - MRO)
```

3. Multilevel Inheritance (Grandparent → Parent → Child)

```
python
```

```
class Grandparent:
    def skills(self):
        return "Basic skills"

class Parent(Grandparent):
    def skills(self):
        return super().skills() + ", Advanced skills"

class Child(Parent):
    def skills(self):
        return super().skills() + ", Expert skills"

c = Child()
print(c.skills()) # Basic skills, Advanced skills, Expert skills
```

4. Hierarchical Inheritance (One parent → Multiple children)

python

```
class Shape:
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2
```

5. Hybrid Inheritance (Combination of types)

python

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass # Diamond problem
```

Method Resolution Order (MRO)

python

```
class A:
    def method(self):
        print("A")

class B(A):
    def method(self):
        print("B")

class C(A):
    def method(self):
        print("C")

class D(B, C):
```

```

pass

d = D()
d.method() # B (follows MRO)
print(D.__mro__) # Shows order: D -> B -> C -> A -> object

```

super() Explained

python

```

class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def info(self):
        return f"Name: {self.name}, Salary: {self.salary}"

class Manager(Employee):
    def __init__(self, name, salary, team_size):
        super().__init__(name, salary) # Call parent constructor
        self.team_size = team_size

    def info(self):
        # Call parent method and add more info
        return super().info() + f", Team Size: {self.team_size}"

m = Manager("Alice", 80000, 5)
print(m.info()) # Name: Alice, Salary: 80000, Team Size: 5

```

PILLAR 3: POLYMORPHISM (Many Forms)

Definition: Same interface, different implementations.

Method Overriding (Runtime Polymorphism)

python

```

class Payment:
    def process(self, amount):
        raise NotImplementedError("Subclass must implement")

class CreditCard(Payment):
    def process(self, amount):

```

```

        return f"Processing ${amount} via Credit Card"

class PayPal(Payment):
    def process(self, amount):
        return f"Processing ${amount} via PayPal"

class Crypto(Payment):
    def process(self, amount):
        return f"Processing ${amount} via Cryptocurrency"

# Polymorphic function
def make_payment(payment_method, amount):
    return payment_method.process(amount)

# Dry Run
payments = [CreditCard(), PayPal(), Crypto()]
for payment in payments:
    print(make_payment(payment, 100))

# Output:
# Processing $100 via Credit Card
# Processing $100 via PayPal
# Processing $100 via Cryptocurrency

```

Duck Typing (Python's way)

```

python

# If it walks like a duck and quacks like a duck, it's a duck!
class Duck:
    def sound(self):
        return "Quack"

class Dog:
    def sound(self):
        return "Woof"

class Car:
    def horn(self):
        return "Beep"

def make_sound(obj):
    # No type checking - just try to call sound()
    return obj.sound()

# These work
print(make_sound(Duck())) # Quack
print(make_sound(Dog())) # Woof

```

```
# This fails (Car doesn't have sound method)
# print(make_sound(Car())) # AttributeError

# Python way: Check if method exists
def make_sound_safe(obj):
    if hasattr(obj, 'sound'):
        return obj.sound()
    return "Object can't make sound"

print(make_sound_safe(Car())) # Object can't make sound
```

Method Overloading (Compile-time Polymorphism)

python

```
# Python doesn't support traditional overloading
# Use default arguments or *args instead

class Calculator:
    # Approach 1: Default arguments
    def add(self, a, b, c=0):
        return a + b + c

    # Approach 2: Variable arguments
    def sum(self, *args):
        return sum(args)

    # Approach 3: Type checking
    def multiply(self, a, b):
        if isinstance(a, int) and isinstance(b, int):
            return a * b
        elif isinstance(a, str) and isinstance(b, int):
            return a * b
        else:
            return "Invalid types"

calc = Calculator()
print(calc.add(5, 3)) # 8
print(calc.add(5, 3, 2)) # 10
print(calc.sum(1, 2, 3)) # 6
print(calc.multiply(5, 3)) # 15
print(calc.multiply("Hi", 3)) # HiHiHi
```

Operator Overloading

python

```

class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):          # +
        return Vector(self.x + other.x, self.y + other.y)

    def __sub__(self, other):          # -
        return Vector(self.x - other.x, self.y - other.y)

    def __mul__(self, scalar):         # *
        return Vector(self.x * scalar, self.y * scalar)

    def __eq__(self, other):           # ==
        return self.x == other.x and self.y == other.y

    def __str__(self):                 # print()
        return f"({self.x}, {self.y})"

    def __repr__(self):                # representation
        return f"Vector({self.x}, {self.y})"

v1 = Vector(2, 3)
v2 = Vector(4, 5)
print(v1 + v2)  # (6, 8)
print(v1 - v2)  # (-2, -2)
print(v1 * 3)   # (6, 9)
print(v1 == Vector(2, 3))  # True

```

Complete Magic Methods List

python

```

class MagicDemo:
    def __init__(self, value):
        self.value = value

    # Arithmetic
    def __add__(self, other): return self.value + other.value # +
    def __sub__(self, other): return self.value - other.value # -
    def __mul__(self, other): return self.value * other.value # *
    def __truediv__(self, other): return self.value / other.value # /
    def __floordiv__(self, other): return self.value // other.value # //
    def __mod__(self, other): return self.value % other.value # %

```

```

def __pow__(self, other): return self.value ** other.value # **

# Comparison
def __eq__(self, other): return self.value == other.value # ==
def __ne__(self, other): return self.value != other.value # !=
def __lt__(self, other): return self.value < other.value # <
def __le__(self, other): return self.value <= other.value # <=
def __gt__(self, other): return self.value > other.value # >
def __ge__(self, other): return self.value >= other.value # >=

# Others
def __len__(self): return len(str(self.value)) # len()
def __str__(self): return str(self.value) # str()
def __bool__(self): return bool(self.value) # bool()
def __call__(self, x): return self.value * x # object()
def __getitem__(self, key): return self.value[key] # indexing
def __setitem__(self, key, value): self.value[key] = value # setting
def __delitem__(self, key): del self.value[key] # deleting
def __iter__(self): return iter(self.value) # iteration
def __contains__(self, item): return item in self.value # in

```

PILLAR 4: ABSTRACTION (Hiding Complexity)

Definition: Show only essential features, hide implementation details.

Abstract Classes & Methods

```

python

from abc import ABC, abstractmethod

class Vehicle(ABC): # Abstract class
    def __init__(self, brand):
        self.brand = brand

    @abstractmethod
    def start(self):
        """Every vehicle must have a start method"""
        pass

    @abstractmethod
    def stop(self):
        """Every vehicle must have a stop method"""
        pass

```

```

def display(self): # Concrete method (optional)
    return f"Vehicle brand: {self.brand}"

class Car(Vehicle):
    def start(self):
        return f"{self.brand} car started with key"

    def stop(self):
        return f"{self.brand} car stopped"

class Bike(Vehicle):
    def start(self):
        return f"{self.brand} bike started with kick"

    def stop(self):
        return f"{self.brand} bike stopped"

class ElectricCar(Vehicle):
    def start(self):
        return f"{self.brand} electric car started silently"

    def stop(self):
        return f"{self.brand} electric car stopped"

# Usage
car = Car("Toyota")
bike = Bike("Honda")
ev = ElectricCar("Tesla")

# All vehicles can start/stop, but implementation differs
print(car.start()) # Toyota car started with key
print(bike.start()) # Honda bike started with kick
print(ev.start()) # Tesla electric car started silently

# vehicle = Vehicle("Test") # ERROR! Can't instantiate abstract class

```

Real-World Abstraction Example

```

python

from abc import ABC, abstractmethod

class Database(ABC):
    """Abstract interface for all databases"""

    @abstractmethod
    def connect(self):

```

```
pass

@abstractmethod
def execute(self, query):
    pass

@abstractmethod
def close(self):
    pass

class MySQLDatabase(Database):
    def connect(self):
        print("Connecting to MySQL...")
        return "MySQL connected"

    def execute(self, query):
        print(f"MySQL executing: {query}")
        return "MySQL result"

    def close(self):
        print("MySQL closed")

class PostgreSQLDatabase(Database):
    def connect(self):
        print("Connecting to PostgreSQL...")
        return "PostgreSQL connected"

    def execute(self, query):
        print(f"PostgreSQL executing: {query}")
        return "PostgreSQL result"

    def close(self):
        print("PostgreSQL closed")

class Application:
    def __init__(self, database):
        self.db = database
        self.db.connect()

    def run_query(self, query):
        return self.db.execute(query)

    def shutdown(self):
        self.db.close()

# Client code doesn't care which database is used
app1 = Application(MySQLDatabase())
print(app1.run_query("SELECT * FROM users"))
```



```
app1.shutdown()

app2 = Application(PostgreSQLDatabase())
print(app2.run_query("SELECT * FROM products"))
app2.shutdown()
```

● ADVANCED CONCEPTS {#advanced}

1. Decorators in OOP

python

```
class Logger:
    @staticmethod
    def log(func):
        """Decorator to log method calls"""
        def wrapper(*args, **kwargs):
            print(f"Calling {func.__name__}")
            result = func(*args, **kwargs)
            print(f"{func.__name__} returned {result}")
            return result
        return wrapper

class Calculator:
    @Logger.log
    def add(self, a, b):
        return a + b

    @Logger.log
    def multiply(self, a, b):
        return a * b

calc = Calculator()
calc.add(5, 3)      # Logs: Calling add, add returned 8
calc.multiply(4, 6) # Logs: Calling multiply, multiply returned 24
```

2. Properties (Getters/Setters)

python

```
class Employee:
    def __init__(self, name, salary):
```

```

        self._name = name
        self._salary = salary
        self._email = f"{name.lower()}@company.com"

    @property
    def name(self): # Getter
        return self._name

    @name.setter
    def name(self, value): # Setter
        if not value:
            raise ValueError("Name cannot be empty")
        self._name = value
        self._email = f"{value.lower()}@company.com"

    @property
    def salary(self):
        return self._salary

    @salary.setter
    def salary(self, value):
        if value < 0:
            raise ValueError("Salary cannot be negative")
        self._salary = value

    @property
    def annual_salary(self): # Computed property
        return self._salary * 12

    @property
    def email(self): # Read-only property
        return self._email

emp = Employee("John", 50000)
print(emp.name)          # John
print(emp.salary)         # 50000
print(emp.annual_salary) # 600000
print(emp.email)          # john@company.com

emp.name = "Jane"        # Updates email automatically
print(emp.email)          # jane@company.com
# emp.email = "test"      # ERROR! No setter

```

3. Class Methods vs Static Methods

python

```

class Person:
    species = "Homo sapiens"
    population = 0

    def __init__(self, name, age):
        self.name = name
        self.age = age
        Person.population += 1

    # Instance method - uses self
    def greet(self):
        return f"Hi, I'm {self.name}, {self.age} years old"

    # Class method - uses cls (class)
    @classmethod
    def get_population(cls):
        return f"Total people: {cls.population}"

    @classmethod
    def create_anonymous(cls):
        return cls("Anonymous", 0) # Factory method

    # Static method - no self or cls
    @staticmethod
    def is_adult(age):
        return age >= 18

    @staticmethod
    def validate_name(name):
        return bool(name and name.strip())

# Instance methods
p1 = Person("Alice", 25)
print(p1.greet()) # Hi, I'm Alice, 25 years old

# Class methods
print(Person.get_population()) # Total people: 1
p2 = Person.create_anonymous()
print(p2.greet()) # Hi, I'm Anonymous, 0 years old

# Static methods
print(Person.is_adult(20)) # True
print(Person.validate_name("")) # False

```

4. Composition vs Inheritance

```
python

# Inheritance: IS-A relationship
class Animal:
    pass

class Dog(Animal): # Dog IS-A Animal
    pass

# Composition: HAS-A relationship
class Engine:
    def start(self):
        return "Engine running"

class Wheels:
    def rotate(self):
        return "Wheels spinning"

class Car:
    def __init__(self):
        self.engine = Engine() # Car HAS-A Engine
        self.wheels = Wheels() # Car HAS-A Wheels

    def drive(self):
        return f"{self.engine.start()} and {self.wheels.rotate()}"

# When to use which?
# Inheritance: "X is a Y" (Dog is an Animal)
# Composition: "X has a Y" (Car has an Engine)

# Composition provides more flexibility
class ElectricEngine(Engine):
    def start(self):
        return "Electric motor running"

class Tesla(Car):
    def __init__(self):
        self.engine = ElectricEngine() # Swap engine easily
        self.wheels = Wheels()

tesla = Tesla()
print(tesla.drive()) # Electric motor running and Wheels spinning
```

5. Mixins

```
python
```

```

# Mixins: Reusable pieces of functionality
class FlyMixin:
    def fly(self):
        return f"{self.name} is flying"

    def land(self):
        return f"{self.name} landed"

class SwimMixin:
    def swim(self):
        return f"{self.name} is swimming"

    def dive(self):
        return f"{self.name} dived"

class WalkMixin:
    def walk(self):
        return f"{self.name} is walking"

# Use mixins to compose behavior
class Duck(WalkMixin, SwimMixin, FlyMixin):
    def __init__(self, name):
        self.name = name

    def quack(self):
        return "Quack!"

class Penguin(WalkMixin, SwimMixin):
    def __init__(self, name):
        self.name = name

duck = Duck("Donald")
print(duck.fly())    # Donald is flying
print(duck.swim())   # Donald is swimming
print(duck.walk())   # Donald is walking

penguin = Penguin("Pingu")
print(penguin.swim()) # Pingu is swimming
# print(penguin.fly()) # ERROR! Penguin can't fly

```

6. Generators in OOP

python

```

class Fibonacci:
    def __init__(self, n):

```

```

        self.n = n
        self.a, self.b = 0, 1
        self.count = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.count >= self.n:
            raise StopIteration
        result = self.a
        self.a, self.b = self.b, self.a + self.b
        self.count += 1
        return result

# Or use generator function
def fibonacci_generator(n):
    a, b = 0, 1
    for _ in range(n):
        yield a
        a, b = b, a + b

fib = Fibonacci(10)
for num in fib:
    print(num) # 0, 1, 1, 2, 3, 5, 8, 13, 21, 34

for num in fibonacci_generator(5):
    print(num) # 0, 1, 1, 2, 3

```

7. Context Managers

python

```

class FileManager:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode
        self.file = None

    def __enter__(self):
        print(f"Opening file: {self.filename}")
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Closing file")

```

```

    if self.file:
        self.file.close()
    if exc_type:
        print(f"Exception occurred: {exc_val}")
    return False # Don't suppress exceptions

# Usage
with FileManager("test.txt", "w") as f:
    f.write("Hello World!")

# Equivalent to:
# f = open("test.txt", "w")
# f.write("Hello World!")
# f.close()

```

8. Metaclasses

python

```

# Metaclass: Class of a class
class Meta(type):
    def __new__(cls, name, bases, attrs):
        # Modify class before creation
        attrs['version'] = "1.0"
        attrs['created_by'] = cls.__name__
        return super().__new__(cls, name, bases, attrs)

    def __call__(cls, *args, **kwargs):
        print(f"Creating instance of {cls.__name__}")
        return super().__call__(*args, **kwargs)

# Use metaclass
class MyClass(metaclass=Meta):
    def __init__(self, value):
        self.value = value

obj = MyClass(10) # Creating instance of MyClass
print(obj.version) # 1.0 (added by metaclass)
print(obj.created_by) # Meta (added by metaclass)

```

9. Descriptors

python

```

class ValidatedString:
    def __init__(self, min_length=0, max_length=100):
        self.min_length = min_length
        self.max_length = max_length

    def __set_name__(self, owner, name):
        self.name = name

    def __get__(self, instance, owner):
        if instance is None:
            return self
        return instance.__dict__.get(self.name, "")

    def __set__(self, instance, value):
        if not isinstance(value, str):
            raise TypeError(f"{self.name} must be a string")
        if len(value) < self.min_length:
            raise ValueError(f"{self.name} too short")
        if len(value) > self.max_length:
            raise ValueError(f"{self.name} too long")
        instance.__dict__[self.name] = value

class User:
    name = ValidatedString(min_length=2, max_length=50)
    password = ValidatedString(min_length=8, max_length=100)

    def __init__(self, name, password):
        self.name = name
        self.password = password

user = User("John", "secure123")
print(user.name) # John
# user.name = "A" # ValueError: name too short

```

● DESIGN PATTERNS {#patterns}

1. Singleton Pattern

python

```

class Singleton:
    _instance = None

```



```

def __new__(cls, *args, **kwargs):
    if cls._instance is None:
        cls._instance = super().__new__(cls)
    return cls._instance

def __init__(self, value):
    self.value = value

# Test
s1 = Singleton("First")
s2 = Singleton("Second")
print(s1 is s2) # True (same object)
print(s1.value) # Second (updated)
print(s2.value) # Second

```

2. Factory Pattern

python

```

from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow!"

class Duck(Animal):
    def speak(self):
        return "Quack!"

class AnimalFactory:
    @staticmethod
    def create_animal(animal_type):
        if animal_type == "dog":
            return Dog()
        elif animal_type == "cat":
            return Cat()
        elif animal_type == "duck":

```

```

        return Duck()
    else:
        raise ValueError(f"Unknown animal: {animal_type}")

# Usage
factory = AnimalFactory()
animals = ["dog", "cat", "duck"]
for animal_type in animals:
    animal = factory.create_animal(animal_type)
    print(f"{animal_type}: {animal.speak()}")

```

3. Observer Pattern

python

```

class Subject:
    def __init__(self):
        self._observers = []
        self._state = None

    def attach(self, observer):
        if observer not in self._observers:
            self._observers.append(observer)

    def detach(self, observer):
        self._observers.remove(observer)

    def notify(self):
        for observer in self._observers:
            observer.update(self)

    @property
    def state(self):
        return self._state

    @state.setter
    def state(self, value):
        self._state = value
        self.notify()

class Observer:
    def update(self, subject):
        print(f"{self.__class__.__name__} received state: {subject.state}")

class Logger(Observer):
    def update(self, subject):

```

```
print(f"LOG: State changed to {subject.state}")

class EmailNotifier(Observer):
    def update(self, subject):
        print(f"EMAIL: Sending notification for state {subject.state}")

# Usage
subject = Subject()
subject.attach(Logger())
subject.attach(EmailNotifier())

subject.state = "Started"
subject.state = "Processing"
subject.state = "Completed"
```

4. Decorator Pattern

python

```
class Coffee:
    def cost(self):
        return 5

    def description(self):
        return "Coffee"

class CoffeeDecorator:
    def __init__(self, coffee):
        self._coffee = coffee

    def cost(self):
        return self._coffee.cost()

    def description(self):
        return self._coffee.description()

class Milk(CoffeeDecorator):
    def cost(self):
        return self._coffee.cost() + 1

    def description(self):
        return self._coffee.description() + ", Milk"

class Sugar(CoffeeDecorator):
    def cost(self):
        return self._coffee.cost() + 0.5
```

```

    def description(self):
        return self._coffee.description() + ", Sugar"

class WhippedCream(CoffeeDecorator):
    def cost(self):
        return self._coffee.cost() + 1.5

    def description(self):
        return self._coffee.description() + ", Whipped Cream"

# Usage
coffee = Coffee()
print(coffee.description(), "({})".format(coffee.cost()))

coffee_with_milk = Milk(coffee)
print(coffee_with_milk.description(), "({})".format(coffee_with_milk.cost()))

fancy_coffee = WhippedCream(Sugar(Milk(coffee)))
print(fancy_coffee.description(), "({})".format(fancy_coffee.cost()))

```

BEST PRACTICES {#practices}

1. SOLID Principles

S - Single Responsibility

python

```

# BAD
class Report:
    def generate(self):
        pass
    def save_to_file(self):
        pass
    def send_email(self):
        pass

# GOOD
class ReportGenerator:
    def generate(self):
        pass

class ReportSaver:

```

```

def save(self, report):
    pass

class EmailSender:
    def send(self, report):
        pass

```

O - Open/Closed

python

```

# BAD
class AreaCalculator:
    def calculate(self, shape):
        if shape.type == "circle":
            return 3.14 * shape.radius ** 2
        elif shape.type == "square":
            return shape.side ** 2

# GOOD
class Shape:
    def area(self):
        pass

class Circle(Shape):
    def area(self):
        return 3.14 * self.radius ** 2

class Square(Shape):
    def area(self):
        return self.side ** 2

```

L - Liskov Substitution

python

```

# BAD - Violates LSP
class Bird:
    def fly(self):
        pass

class Penguin(Bird): # Penguin can't fly!
    def fly(self):
        raise NotImplementedError

# GOOD

```

```
class Bird:
    pass

class FlyingBird(Bird):
    def fly(self):
        pass

class Penguin(Bird):
    def swim(self):
        pass
```

I - Interface Segregation

python

```
# BAD - Fat interface
class Worker:
    def work(self):
        pass
    def eat(self):
        pass
    def sleep(self):
        pass

# GOOD - Segregated interfaces
class Workable:
    def work(self):
        pass

class Eatable:
    def eat(self):
        pass

class Sleepable:
    def sleep(self):
        pass

class Robot(Workable):
    def work(self):
        pass

class Human(Workable, Eatable, Sleepable):
    def work(self):
        pass
    def eat(self):
        pass
    def sleep(self):
        pass
```

```
def sleep(self):  
    pass
```

D - Dependency Inversion

python

BAD - High Level depends on Low Level

```
class Email:  
    def send(self, message):  
        pass
```

```
class Notification:  
    def __init__(self):  
        self.email = Email()  
  
    def notify(self, message):  
        self.email.send(message)
```

GOOD - Both depend on abstraction

```
class MessageSender:  
    def send(self, message):  
        pass
```

```
class Email(MessageSender):  
    def send(self, message):  
        pass
```

```
class SMS(MessageSender):  
    def send(self, message):  
        pass
```

```
class Notification:  
    def __init__(self, sender: MessageSender):  
        self.sender = sender  
  
    def notify(self, message):  
        self.sender.send(message)
```

2. Naming Conventions

python

```
class StudentInfo: # Class: CamelCase  
    """Class docstring"""
```

```

# Constants: UPPER_CASE
MAX_STUDENTS = 100
DEFAULT_GRADE = "A"

def __init__(self, name, age):
    # Instance variables: snake_case
    self.name = name
    self._age = age # Protected: _single_leading_underscore
    self.__id = 0 # Private: __double_leading_underscore

def get_details(self): # Method: snake_case
    return f"{self.name}, {self._age}"

def _internal_method(self): # Protected method
    pass

def __private_method(self): # Private method
    pass

# Function: snake_case
def calculate_average(grades):
    pass

# Variable: snake_case
student_count = 10

```

3. Type Hints

python

```

from typing import List, Dict, Optional, Union, Tuple

class Book:
    def __init__(self, title: str, author: str, pages: int):
        self.title = title
        self.author = author
        self.pages = pages

    def get_info(self) -> str:
        return f"{self.title} by {self.author}"

    def is_long(self, threshold: int = 300) -> bool:
        return self.pages > threshold

class Library:
    def __init__(self, books: Optional[List[Book]] = None):

```



```

self.books = books or []

def add_book(self, book: Book) -> None:
    self.books.append(book)

def find_by_author(self, author: str) -> List[Book]:
    return [b for b in self.books if b.author == author]

def get_stats(self) -> Dict[str, Union[int, float]]:
    if not self.books:
        return {"total": 0, "avg_pages": 0}
    total_pages = sum(b.pages for b in self.books)
    return {
        "total": len(self.books),
        "avg_pages": total_pages / len(self.books)
    }

```

REAL-WORLD EXAMPLES {#real-world}

1. E-Commerce System

python

```

from abc import ABC, abstractmethod
from typing import List
from datetime import datetime
import uuid

class User:
    def __init__(self, name: str, email: str):
        self.id = str(uuid.uuid4())
        self.name = name
        self.email = email
        self.cart = ShoppingCart()
        self.orders = []

    def add_to_cart(self, product, quantity=1):
        self.cart.add_item(product, quantity)

    def checkout(self):
        if self.cart.items:
            order = Order(self, self.cart)
            self.orders.append(order)
            self.cart.clear()

```

```

        return order
    return None

class Product:
    def __init__(self, name: str, price: float, stock: int):
        self.id = str(uuid.uuid4())
        self.name = name
        self.price = price
        self.stock = stock

    def reduce_stock(self, quantity: int):
        if quantity > self.stock:
            raise ValueError(f"Insufficient stock for {self.name}")
        self.stock -= quantity

class ShoppingCart:
    def __init__(self):
        self.items = {} # product_id -> (product, quantity)

    def add_item(self, product: Product, quantity: int):
        if product.id in self.items:
            current_qty = self.items[product.id][1]
            self.items[product.id] = (product, current_qty + quantity)
        else:
            self.items[product.id] = (product, quantity)

    def remove_item(self, product_id: str):
        if product_id in self.items:
            del self.items[product_id]

    def get_total(self):
        return sum(p.price * qty for p, qty in self.items.values())

    def clear(self):
        self.items.clear()

class Payment(ABC):
    @abstractmethod
    def process(self, amount: float) -> bool:
        pass

class CreditCardPayment(Payment):
    def __init__(self, card_number: str, expiry: str, cvv: str):
        self.card_number = card_number
        self.expiry = expiry
        self.cvv = cvv

    def process(self, amount: float) -> bool:

```

```

    # Simulate payment processing
    print(f"Processing ${amount} via Credit Card ending in {self.card_number[-4:]}")
    return True

class PayPalPayment(Payment):
    def __init__(self, email: str):
        self.email = email

    def process(self, amount: float) -> bool:
        print(f"Processing ${amount} via PayPal account {self.email}")
        return True

class Order:
    def __init__(self, user: User, cart: ShoppingCart):
        self.id = str(uuid.uuid4())
        self.user = user
        self.items = dict(cart.items)
        self.total = cart.get_total()
        self.date = datetime.now()
        self.status = "pending"

    def pay(self, payment: Payment):
        if payment.process(self.total):
            self.status = "paid"
            for product, qty in self.items.values():
                product.reduce_stock(qty)
            return True
        return False

# Usage
# Create users
alice = User("Alice", "alice@email.com")
bob = User("Bob", "bob@email.com")

# Add products
laptop = Product("Laptop", 999.99, 10)
phone = Product("Phone", 599.99, 20)
book = Product("Book", 29.99, 50)

# Alice shops
alice.add_to_cart(laptop, 1)
alice.add_to_cart(book, 2)

# Checkout
order = alice.checkout()
if order:
    payment = CreditCardPayment("1234567890123456", "12/25", "123")
    if order.pay(payment):

```

```

print(f"Order {order.id} paid successfully!")
print(f"Total: ${order.total}")
print(f"Items: {len(order.items)}")

# Bob shops differently
bob.add_to_cart(phone, 1)
order_bob = bob.checkout()
if order_bob:
    payment_bob = PayPalPayment("bob@email.com")
    order_bob.pay(payment_bob)
    print(f"Bob's order total: ${order_bob.total}")

```

2. Game System

python

```

from abc import ABC, abstractmethod
import random

class Character(ABC):
    def __init__(self, name, health=100, attack=10, defense=5):
        self.name = name
        self.max_health = health
        self.health = health
        self.attack = attack
        self.defense = defense
        self.level = 1
        self.experience = 0
        self.inventory = []

    @abstractmethod
    def special_ability(self):
        pass

    def take_damage(self, damage):
        actual_damage = max(0, damage - self.defense)
        self.health -= actual_damage
        return actual_damage

    def is_alive(self):
        return self.health > 0

    def attack_enemy(self, enemy):
        damage = self.attack + random.randint(-2, 5)
        actual_damage = enemy.take_damage(damage)
        return f"{self.name} deals {actual_damage} damage to {enemy.name}"

```

```
def gain_experience(self, exp):
    self.experience += exp
    if self.experience >= self.level * 100:
        self.level_up()

def level_up(self):
    self.level += 1
    self.max_health += 20
    self.health = self.max_health
    self.attack += 5
    self.defense += 3
    print(f"{self.name} leveled up to {self.level}!")

class Warrior(Character):
    def __init__(self, name):
        super().__init__(name, health=150, attack=15, defense=10)

    def special_ability(self):
        self.attack *= 2
        return f"{self.name} uses Rage! Attack doubled for this turn!"

class Mage(Character):
    def __init__(self, name):
        super().__init__(name, health=80, attack=25, defense=3)
        self.mana = 100

    def special_ability(self):
        if self.mana >= 50:
            self.mana -= 50
            return f"{self.name} casts Fireball! Massive damage!"
        return "Not enough mana"

class Archer(Character):
    def __init__(self, name):
        super().__init__(name, health=100, attack=20, defense=5)
        self.arrows = 20

    def special_ability(self):
        if self.arrows >= 5:
            self.arrows -= 5
            self.attack += 10
            return f"{self.name} uses Multi-shot! Attack increased!"
        return "Not enough arrows"

class Enemy:
    def __init__(self, name, health, attack, defense, exp_reward):
        self.name = name
```

```

        self.health = health
        self.attack = attack
        self.defense = defense
        self.exp_reward = exp_reward

    def take_damage(self, damage):
        actual_damage = max(0, damage - self.defense)
        self.health -= actual_damage
        return actual_damage

    def is_alive(self):
        return self.health > 0

class Game:
    def __init__(self):
        self.player = None
        self.enemies = []

    def create_player(self):
        print("Choose your class:")
        print("1. Warrior (Tanky)")
        print("2. Mage (High damage)")
        print("3. Archer (Balanced)")

        choice = input("Enter choice (1-3): ")
        name = input("Enter character name: ")

        if choice == "1":
            self.player = Warrior(name)
        elif choice == "2":
            self.player = Mage(name)
        elif choice == "3":
            self.player = Archer(name)
        else:
            print("Invalid choice, defaulting to Warrior")
            self.player = Warrior(name)

    def spawn_enemy(self):
        enemy_types = [
            ("Goblin", 50, 8, 2, 50),
            ("Orc", 80, 12, 5, 80),
            ("Troll", 120, 10, 8, 120),
            ("Dragon", 200, 25, 10, 300)
        ]
        enemy_data = random.choice(enemy_types)
        enemy = Enemy(*enemy_data)
        print(f"A wild {enemy.name} appears!")
        return enemy

```

```

def battle(self, enemy):
    print(f"\n=== Battle with {enemy.name} ===")
    battle_log = []

    while self.player.is_alive() and enemy.is_alive():
        print(f"\n{self.player.name}: {self.player.health}/{self.player.max_health} HP")

        print(f"{enemy.name}: {enemy.health} HP")

        print("\nActions:")
        print("1. Attack")
        print("2. Special Ability")
        print("3. Defend (reduce damage)")

        choice = input("Choose action: ")

        if choice == "1":
            result = self.player.attack_enemy(enemy)
            battle_log.append(result)
            print(result)

        elif choice == "2":
            result = self.player.special_ability()
            battle_log.append(result)
            print(result)
            if "Fireball" in result or "Rage" in result or "Multi-shot" in result:
                result = self.player.attack_enemy(enemy)
                battle_log.append(result)
                print(result)

        elif choice == "3":
            self.player.defense += 5
            print(f"{self.player.name} defends!")

        if enemy.is_alive():
            enemy_damage = enemy.attack + random.randint(-3, 3)
            actual_damage = self.player.take_damage(enemy_damage)
            print(f"{enemy.name} deals {actual_damage} damage")

    if self.player.is_alive():
        print(f"\nVictory! {self.player.name} defeated {enemy.name}!")
        self.player.gain_experience(enemy.exp_reward)
        print(f"Gained {enemy.exp_reward} experience!")
        return True
    else:
        print(f"\nGame Over! {self.player.name} was defeated...")
        return False

```

```

def play(self):
    print("=== Welcome to RPG Adventure ===")
    self.create_player()
    print(f"\nWelcome, {self.player.name}!")

    round_num = 1
    while self.player.is_alive():
        print(f"\n--- Round {round_num} ---")
        enemy = self.spawn_enemy()
        if not self.battle(enemy):
            break
        round_num += 1

        if input("\nContinue adventuring? (y/n): ").lower() != 'y':
            break

    print(f"\nGame Over! {self.player.name} reached Level {self.player.level}")

# Start game
# game = Game()
# game.play()

```

3. Data Processing Pipeline

```

python

from abc import ABC, abstractmethod
from typing import Any, List, Dict
import json
import csv
from datetime import datetime

class DataSource(ABC):
    @abstractmethod
    def read(self) -> List[Dict]:
        pass

class DataProcessor(ABC):
    @abstractmethod
    def process(self, data: List[Dict]) -> List[Dict]:
        pass

class DataExporter(ABC):
    @abstractmethod
    def export(self, data: List[Dict]) -> str:

```



```
pass

class JSONDataSource(DataSource):
    def __init__(self, filename: str):
        self.filename = filename

    def read(self) -> List[Dict]:
        with open(self.filename, 'r') as f:
            return json.load(f)

class CSVDataSource(DataSource):
    def __init__(self, filename: str):
        self.filename = filename

    def read(self) -> List[Dict]:
        data = []
        with open(self.filename, 'r') as f:
            reader = csv.DictReader(f)
            for row in reader:
                data.append(row)
        return data

class CleanDataProcessor(DataProcessor):
    def process(self, data: List[Dict]) -> List[Dict]:
        cleaned = []
        for item in data:
            # Remove empty values
            cleaned_item = {k: v for k, v in item.items() if v and v.strip()}

            # Convert numeric strings
            for key, value in cleaned_item.items():
                if isinstance(value, str):
                    if value.isdigit():
                        cleaned_item[key] = int(value)
                    elif self._is_float(value):
                        cleaned_item[key] = float(value)

            # Standardize dates
            if 'date' in cleaned_item:
                cleaned_item['date'] = self._parse_date(cleaned_item['date'])

            cleaned.append(cleaned_item)
        return cleaned

    def _is_float(self, value: str) -> bool:
        try:
            float(value)
            return True
```

```
except ValueError:
    return False

def _parse_date(self, date_str: str) -> str:
    try:
        date = datetime.strptime(date_str, "%Y-%m-%d")
        return date.strftime("%B %d, %Y")
    except:
        return date_str

class CalculateStatsProcessor(DataProcessor):
    def __init__(self, numeric_fields: List[str]):
        self.numeric_fields = numeric_fields

    def process(self, data: List[Dict]) -> List[Dict]:
        stats = {}
        for field in self.numeric_fields:
            values = []
            for item in data:
                if field in item and isinstance(item[field], (int, float)):
                    values.append(item[field])

            if values:
                stats[f"{field}_avg"] = round(sum(values) / len(values), 2)
                stats[f"{field}_min"] = min(values)
                stats[f"{field}_max"] = max(values)

        return [{"stats": stats, "data": data}]

class JSONExporter(DataExporter):
    def export(self, data: List[Dict]) -> str:
        return json.dumps(data, indent=2)

class DataPipeline:
    def __init__(self):
        self.sources = []
        self.processors = []
        self.exporters = []

    def add_source(self, source: DataSource):
        self.sources.append(source)

    def add_processor(self, processor: DataProcessor):
        self.processors.append(processor)

    def add_exporter(self, exporter: DataExporter):
        self.exporters.append(exporter)
```

```
def execute(self) -> str:
    data = []
    for source in self.sources:
        data.extend(source.read())

    for processor in self.processors:
        data = processor.process(data)

    result = ""
    for exporter in self.exporters:
        result += exporter.export(data) + "\n"

    return result

# Usage
pipeline = DataPipeline()

# Add source
pipeline.add_source(JSONDataSource("data.json"))

# Add processors
pipeline.add_processor(CleanDataProcessor())
pipeline.add_processor(CalculateStatsProcessor(["price", "quantity"]))

# Add exporter
pipeline.add_exporter(JSONExporter())

# Execute pipeline
# result = pipeline.execute()
# print(result)
```

 QUICK REFERENCE CHEAT SHEET

OOP Concepts Comparison

Concept	Purpose	Keyword	Example
Class	Blueprint	class	class Car:
Object	Instance	object()	my_car = Car()
Constructor	Initialize	__init__	def __init__(self):

Concept	Purpose	Keyword	Example
Inheritance	Reuse	<code>super()</code>	<code>class SUV(Car):</code>
Polymorphism	Many forms	Override	<code>def start(self):</code>
Encapsulation	Hide data	<code>__</code>	<code>self.__balance</code>
Abstraction	Hide complexity	<code>@abstractmethod</code>	<code>@abstractmethod</code>
Property	Getter/Setter	<code>@property</code>	<code>@property</code>
Static Method	No instance	<code>@staticmethod</code>	<code>@staticmethod</code>
Class Method	Class level	<code>@classmethod</code>	<code>@classmethod</code>

Common Magic Methods

Method	Operation	Example
<code>__init__</code>	Constructor	<code>obj = MyClass()</code>
<code>__str__</code>	String for print	<code>print(obj)</code>
<code>__repr__</code>	Developer string	<code>repr(obj)</code>
<code>__len__</code>	Length	<code>len(obj)</code>
<code>__getitem__</code>	Indexing	<code>obj[0]</code>
<code>__setitem__</code>	Setting	<code>obj[0] = value</code>
<code>__iter__</code>	Iteration	<code>for x in obj:</code>
<code>__contains__</code>	Membership	<code>x in obj</code>
<code>__call__</code>	Callable	<code>obj()</code>
<code>__add__</code>	Addition	<code>obj1 + obj2</code>
<code>__eq__</code>	Equality	<code>obj1 == obj2</code>

 PRO TIPS

1. **Favor composition over inheritance** - More flexible
2. **Keep classes focused** - Single responsibility
3. **Use properties not direct access** - Better control
4. **Document your classes** - Help others understand
5. **Write unit tests** - Verify behavior
6. **Use type hints** - Better IDE support
7. **Design for extension** - Open/Closed principle
8. **Don't repeat yourself** - DRY principle
9. **Keep it simple** - KISS principle
10. **Test edge cases** - Empty lists, None values

 COMMON INTERVIEW QUESTIONS

1. **What are the 4 pillars of OOP?**
 - Encapsulation, Inheritance, Polymorphism, Abstraction
2. **What's the difference between class and object?**
 - Class = blueprint, Object = instance
3. **What is `self` in Python?**
 - Reference to current instance
4. **What is Method Resolution Order (MRO)?**
 - Order Python searches for methods in inheritance
5. **What's the difference between `__str__` and `__repr__`?**
 - `__str__` = user-friendly, `__repr__` = developer-friendly
6. **What is a decorator in OOP?**
 - Function that modifies another function's behavior
7. **What's the difference between abstract class and interface?**
 - Abstract class can have implementation, interface is pure contract
8. **What is dependency injection?**
 - Passing dependencies rather than creating them internally

🎉 **Congratulations!** You now have a complete understanding of OOP from basics to advanced concepts!

This response is AI-generated, for reference only.

all oops method and atribute @abstrac

COMPLETE OOP METHODS & ATTRIBUTES GUIDE 🎯

📖 TABLE OF CONTENTS

1. Attributes (Variables)
2. Methods (Functions)
3. Abstract Classes & Methods
4. Special/Advanced Methods
5. Property Decorators
6. Complete Examples

🟢 ATTRIBUTES (Variables) {#attributes}

1. Instance Attributes (Object-specific)

python

```
class Student:
    def __init__(self, name, age, grade):
        # Instance attributes - unique to each object
        self.name = name
        self.age = age
        self.grade = grade
        self.scores = [] # Each student has their own list

    def add_score(self, score):
        self.scores.append(score)

# Each object has its own attributes
s1 = Student("Alice", 20, "A")
s2 = Student("Bob", 22, "B")
```

```

print(s1.name) # Alice
print(s2.name) # Bob
s1.scores.append(95)
s2.scores.append(85)
print(s1.scores) # [95]
print(s2.scores) # [85] - Different!

```

2. Class Attributes (Shared)

python

```

class Student:
    # Class attributes - shared by all instances
    school = "Harvard University"
    student_count = 0
    max_students = 100

    def __init__(self, name, age):
        self.name = name
        self.age = age
        Student.student_count += 1 # Increment shared counter

    @classmethod
    def is_full(cls):
        return cls.student_count >= cls.max_students

# Access via class
print(Student.school) # Harvard University
print(Student.student_count) # 0

# Access via instance
s1 = Student("Alice", 20)
s2 = Student("Bob", 22)
print(Student.student_count) # 2
print(s1.school) # Harvard University
print(s2.school) # Harvard University

# Changing class attribute affects all instances
Student.school = "MIT"
print(s1.school) # MIT
print(s2.school) # MIT

```

3. Private Attributes (Encapsulation)

python

```

class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner          # Public
        self._balance = balance     # Protected (convention)
        self.__password = "1234"   # Private (name mangling)
        self.__transaction_history = [] # Private

    # Getter method
    def get_balance(self):
        return self._balance

    # Setter method with validation
    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
            self.__add_transaction(f"Deposited: ${amount}")
            return True
        return False

    def withdraw(self, amount):
        if 0 < amount <= self._balance:
            self._balance -= amount
            self.__add_transaction(f"Withdrew: ${amount}")
            return True
        return False

    def __add_transaction(self, transaction): # Private method
        self.__transaction_history.append(transaction)

    def get_history(self):
        return self.__transaction_history.copy() # Return copy

# Usage
acc = BankAccount("John", 1000)
print(acc.owner)          # John (public)
print(acc._balance)       # 1000 (protected - can access but shouldn't)
# print(acc.__password)   # ERROR! AttributeError
# print(acc.__transaction_history) # ERROR!

acc.deposit(500)
print(acc.get_balance())  # 1500
print(acc.get_history())  # ['Deposited: $500']

# Name mangling (don't do this!)
print(acc._BankAccount__password) # 1234 (works but DON'T!)

```


4. Types of Attributes Summary

python

```
class Demo:
    # Class attribute (shared)
    class_var = "I belong to class"

    def __init__(self, value):
        # Instance attribute (unique)
        self.instance_var = value

        # Private instance attribute
        self.__private_var = "Secret"

        # Protected instance attribute
        self._protected_var = "Semi-private"

    # Dynamic attribute creation
    def add_attribute(self):
        self.dynamic_var = "Created at runtime"

obj = Demo("Hello")
print(Demo.class_var)      # I belong to class
print(obj.instance_var)    # Hello
obj.add_attribute()
print(obj.dynamic_var)     # Created at runtime
```

METHODS (Functions) {#methods}

1. Instance Methods (Most Common)

python

```
class Calculator:
    def __init__(self, initial_value=0):
        self.value = initial_value

    # Instance method - uses self
    def add(self, number):
        self.value += number
        return self.value

    def subtract(self, number):
```

```

        self.value -= number
    return self.value

    def multiply(self, number):
        self.value *= number
    return self.value

    def divide(self, number):
        if number != 0:
            self.value /= number
            return self.value
        raise ValueError("Cannot divide by zero")

    def reset(self):
        self.value = 0
    return self.value

calc = Calculator(10)
print(calc.add(5))      # 15
print(calc.multiply(2)) # 30
print(calc.subtract(10)) # 20
print(calc.divide(4))   # 5.0
print(calc.reset())     # 0

```

2. Class Methods (@classmethod)

python

```

class Employee:
    # Class attributes
    company = "Google"
    total_employees = 0
    all_employees = []

    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
        Employee.total_employees += 1
        Employee.all_employees.append(self)

    # Class method - receives class as first argument
    @classmethod
    def get_total(cls):
        return f"Total employees: {cls.total_employees}"

    @classmethod

```

```

def get_company_info(cls):
    return f"Company: {cls.company}, Employees: {cls.total_employees}"

@classmethod
def create_from_string(cls, emp_string):
    """Factory method - creates employee from string"""
    name, salary = emp_string.split("-")
    return cls(name, int(salary))

@classmethod
def get_employee_names(cls):
    return [emp.name for emp in cls.all_employees]

# Usage
emp1 = Employee("Alice", 80000)
emp2 = Employee("Bob", 90000)
emp3 = Employee.create_from_string("Charlie-75000")

print(Employee.get_total()) # Total employees: 3
print(Employee.get_company_info()) # Company: Google, Employees: 3
print(Employee.get_employee_names()) # ['Alice', 'Bob', 'Charlie']

```

3. Static Methods (@staticmethod)

python

```

class MathOperations:
    # Static methods - don't need self or cls
    @staticmethod
    def add(x, y):
        return x + y

    @staticmethod
    def subtract(x, y):
        return x - y

    @staticmethod
    def multiply(x, y):
        return x * y

    @staticmethod
    def divide(x, y):
        if y != 0:
            return x / y
        return "Division by zero"

```

```

@staticmethod
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

@staticmethod
def factorial(n):
    if n <= 1:
        return 1
    return n * MathOperations.factorial(n - 1)

# Call without creating object
print(MathOperations.add(10, 5))      # 15
print(MathOperations.multiply(4, 6))  # 24
print(MathOperations.is_prime(17))    # True
print(MathOperations.factorial(5))    # 120

```

4. Private Methods

python

```

class SecureData:
    def __init__(self, data):
        self.__data = data

    # Private method - can only be called inside class
    def __encrypt(self, text):
        return ''.join(chr(ord(c) + 1) for c in text)

    def __decrypt(self, encrypted):
        return ''.join(chr(ord(c) - 1) for c in encrypted)

    def get_data(self, password):
        if self.__authenticate(password):
            encrypted = self.__encrypt(self.__data)
            return self.__decrypt(encrypted)
        return "Access denied"

    def __authenticate(self, password):
        # Private authentication
        return password == "secret123"

```

```

    # Public method that uses private methods
    def show_encrypted(self):
        return self.__encrypt(self.__data)

obj = SecureData("Hello World")
print(obj.show_encrypted()) # Ifmmp!Xpsme
print(obj.get_data("secret123")) # Hello World
# print(obj.__encrypt("test")) # ERROR! Can't call private method

```

5. Protected Methods (Convention)

python

```

class Parent:
    def __init__(self):
        self._protected_var = "Protected"

    def _protected_method(self):
        return "This is protected"

    def public_method(self):
        return self._protected_method() # Can call protected

class Child(Parent):
    def access_protected(self):
        # Can access protected in subclass
        return self._protected_method()

obj = Parent()
print(obj.public_method()) # This is protected
print(obj._protected_method()) # Works (but shouldn't)
print(obj._protected_var) # Protected (convention)

child = Child()
print(child.access_protected()) # This is protected

```

ABSTRACT CLASSES & METHODS {#abstract}

1. Basic Abstract Class

python

```
from abc import ABC, abstractmethod

class Shape(ABC):
    """Abstract base class for all shapes"""

    def __init__(self, color="black"):
        self.color = color
        self._area = None # Cached area

    # Abstract method - must be implemented
    @abstractmethod
    def calculate_area(self):
        """Calculate the area of the shape"""
        pass

    # Abstract method - must be implemented
    @abstractmethod
    def calculate_perimeter(self):
        """Calculate the perimeter of the shape"""
        pass

    # Abstract property - must be implemented
    @property
    @abstractmethod
    def dimensions(self):
        """Return shape dimensions"""
        pass

    # Concrete method - can be used as-is
    def get_color(self):
        return self.color

    def display_info(self):
        return f"Shape color: {self.color}, Area: {self.calculate_area()}"

# Implementing abstract class
class Rectangle(Shape):
    def __init__(self, length, width, color="black"):
        super().__init__(color)
        self.length = length
        self.width = width

    def calculate_area(self):
        self._area = self.length * self.width
        return self._area

    def calculate_perimeter(self):
```

```

        return 2 * (self.length + self.width)

    @property
    def dimensions(self):
        return f"{self.length}x{self.width}"

class Circle(Shape):
    def __init__(self, radius, color="black"):
        super().__init__(color)
        self.radius = radius

    def calculate_area(self):
        self._area = 3.14159 * self.radius ** 2
        return self._area

    def calculate_perimeter(self):
        return 2 * 3.14159 * self.radius

    @property
    def dimensions(self):
        return f"Radius: {self.radius}"

class Square(Rectangle):
    def __init__(self, side, color="black"):
        super().__init__(side, side, color)

    @property
    def dimensions(self):
        return f"Side: {self.length}"

# Usage
# shape = Shape() # ERROR! Can't instantiate abstract class

rect = Rectangle(5, 3, "blue")
print(rect.calculate_area())      # 15
print(rect.calculate_perimeter()) # 16
print(rect.dimensions)           # 5x3
print(rect.display_info())        # Shape color: blue, Area: 15

circle = Circle(4, "red")
print(circle.calculate_area())    # 50.26544
print(circle.dimensions)         # Radius: 4

```

2. Abstract Methods with Implementation

python

```

from abc import ABC, abstractmethod

class Database(ABC):
    @abstractmethod
    def connect(self):
        """Connect to database - must implement"""
        pass

    @abstractmethod
    def execute_query(self, query):
        """Execute query - must implement"""
        pass

    @abstractmethod
    def disconnect(self):
        """Disconnect - must implement"""
        pass

    # Abstract method with default implementation
    def validate_query(self, query):
        """Optional to override, has default implementation"""
        return query.strip().upper().endswith(";")

    def execute_safe(self, query):
        """Concrete method using abstract methods"""
        if self.validate_query(query):
            return self.execute_query(query)
        return "Invalid query"

class MySQLDatabase(Database):
    def __init__(self, host, user, password):
        self.host = host
        self.user = user
        self.password = password
        self.connected = False

    def connect(self):
        self.connected = True
        return f"Connected to MySQL at {self.host}"

    def execute_query(self, query):
        if self.connected:
            return f"MySQL executing: {query}"
        return "Not connected"

    def disconnect(self):
        self.connected = False

```



```

        return "Disconnected from MySQL"

    # Override default validation
    def validate_query(self, query):
        return query.strip().upper().endswith(";") and "DROP" not in query.upper()

class PostgreSQLDatabase(Database):
    def __init__(self, connection_string):
        self.conn_str = connection_string
        self.connected = False

    def connect(self):
        self.connected = True
        return f"Connected to PostgreSQL"

    def execute_query(self, query):
        if self.connected:
            return f"PostgreSQL executing: {query}"
        return "Not connected"

    def disconnect(self):
        self.connected = False
        return "Disconnected from PostgreSQL"

# Usage
mysql = MySQLDatabase("localhost", "admin", "pass")
print(mysql.connect())           # Connected to MySQL at Localhost
print(mysql.execute_query("SELECT * FROM users;")) # MySQL executing...
print(mysql.execute_safe("DROP TABLE users;")) # Invalid query
mysql.disconnect()

postgres = PostgreSQLDatabase("postgres://localhost")
print(postgres.connect())

```

3. Abstract Properties

python

```

from abc import ABC, abstractmethod

class Vehicle(ABC):
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    @property

```

```

@abstractmethod
def max_speed(self):
    """Maximum speed of vehicle - must implement"""
    pass

@property
@abstractmethod
def fuel_type(self):
    """Type of fuel - must implement"""
    pass

@property
@abstractmethod
def passenger_capacity(self):
    """Number of passengers - must implement"""
    pass

@abstractmethod
def start_engine(self):
    pass

@abstractmethod
def stop_engine(self):
    pass

def vehicle_info(self):
    return f"{self.brand} {self.model}"

class Car(Vehicle):
    def __init__(self, brand, model, max_speed, fuel_type, passengers, doors):
        super().__init__(brand, model)
        self._max_speed = max_speed
        self._fuel_type = fuel_type
        self._passengers = passengers
        self.doors = doors
        self.engine_running = False

    @property
    def max_speed(self):
        return self._max_speed

    @property
    def fuel_type(self):
        return self._fuel_type

    @property
    def passenger_capacity(self):
        return self._passengers

```

```

def start_engine(self):
    self.engine_running = True
    return "Car engine started"

def stop_engine(self):
    self.engine_running = False
    return "Car engine stopped"

class ElectricCar(Car):
    def __init__(self, brand, model, max_speed, passengers, doors, battery_capacity):
        super().__init__(brand, model, max_speed, "Electric", passengers, doors)
        self.battery_capacity = battery_capacity

    def start_engine(self):
        self.engine_running = True
        return "Electric car started silently"

    def check_battery(self):
        return f"Battery: {self.battery_capacity} kWh"

# Usage
car = Car("Toyota", "Camry", 220, "Petrol", 5, 4)
print(car.vehicle_info()) # Toyota Camry
print(car.max_speed)      # 220
print(car.fuel_type)      # Petrol
print(car.start_engine()) # Car engine started

tesla = ElectricCar("Tesla", "Model 3", 250, 5, 4, 75)
print(tesla.vehicle_info()) # Tesla Model 3
print(tesla.fuel_type)      # Electric
print(tesla.start_engine()) # Electric car started silently
print(tesla.check_battery()) # Battery: 75 kWh

```

4. Abstract Class with Concrete Methods

```

python

from abc import ABC, abstractmethod

class Logger(ABC):
    """Abstract logger with common functionality"""

    def __init__(self, level="INFO"):
        self.level = level
        self.logs = []

```

```

@abstractmethod
def log(self, message, level):
    """Log a message - must implement"""
    pass

# Concrete methods
def info(self, message):
    self.log(message, "INFO")

def warning(self, message):
    self.log(message, "WARNING")

def error(self, message):
    self.log(message, "ERROR")

def debug(self, message):
    self.log(message, "DEBUG")

def get_logs(self, level=None):
    if level:
        return [log for log in self.logs if log["level"] == level]
    return self.logs

def clear_logs(self):
    self.logs = []
    return "Logs cleared"

class FileLogger(Logger):
    def __init__(self, filename, level="INFO"):
        super().__init__(level)
        self.filename = filename

    def log(self, message, level):
        import datetime
        timestamp = datetime.datetime.now().isoformat()
        log_entry = {
            "timestamp": timestamp,
            "level": level,
            "message": message
        }
        self.logs.append(log_entry)

        # Write to file
        with open(self.filename, "a") as f:
            f.write(f"{timestamp} [{level}] {message}\n")

        return f"Logged to {self.filename}"

```

```

class ConsoleLogger(Logger):
    def log(self, message, level):
        log_entry = {
            "timestamp": "now",
            "level": level,
            "message": message
        }
        self.logs.append(log_entry)

    # Color coding
    colors = {
        "ERROR": "\033[91m", # Red
        "WARNING": "\033[93m", # Yellow
        "INFO": "\033[92m", # Green
        "DEBUG": "\033[94m" # Blue
    }
    color = colors.get(level, "\033[0m")
    print(f"{color}[{level}] {message}\033[0m")

    return f"Printed to console"

# Usage
file_logger = FileLogger("app.log")
file_logger.info("Application started")
file_logger.warning("Low disk space")
file_logger.error("Database connection failed")

console_logger = ConsoleLogger()
console_logger.info("User logged in")
console_logger.warning("Session expiring soon")
console_logger.error("Invalid password attempt")

print(file_logger.get_logs("ERROR")) # Get only error Logs

```

● ADVANCED METHODS {#advanced-methods}

1. Magic Methods (Dunder Methods)

python

```

class AdvancedClass:
    def __init__(self, value):
        self.value = value

```

```
self.data = [1, 2, 3]

# String representation
def __str__(self):
    return f"AdvancedClass({self.value})"

def __repr__(self):
    return f"AdvancedClass({repr(self.value)})"

# Arithmetic operations
def __add__(self, other):
    return AdvancedClass(self.value + other.value)

def __sub__(self, other):
    return AdvancedClass(self.value - other.value)

def __mul__(self, other):
    if isinstance(other, (int, float)):
        return AdvancedClass(self.value * other)
    return AdvancedClass(self.value * other.value)

def __truediv__(self, other):
    if other.value != 0:
        return AdvancedClass(self.value / other.value)
    raise ZeroDivisionError

# Comparison operations
def __eq__(self, other):
    return self.value == other.value

def __lt__(self, other):
    return self.value < other.value

def __le__(self, other):
    return self.value <= other.value

# Container operations
def __len__(self):
    return len(self.data)

def __getitem__(self, index):
    return self.data[index]

def __setitem__(self, index, value):
    self.data[index] = value

def __delitem__(self, index):
    del self.data[index]
```

```

def __contains__(self, item):
    return item in self.data

# Iteration
def __iter__(self):
    return iter(self.data)

# Callable
def __call__(self, multiplier):
    return self.value * multiplier

# Context manager
def __enter__(self):
    print("Entering context")
    return self

def __exit__(self, exc_type, exc_val, exc_tb):
    print("Exiting context")
    if exc_type:
        print(f"Exception: {exc_val}")
    return False

# Usage
obj1 = AdvancedClass(10)
obj2 = AdvancedClass(5)

print(str(obj1))           # AdvancedClass(10)
print(repr(obj1))          # AdvancedClass(10)
print(obj1 + obj2)         # AdvancedClass(15)
print(obj1 - obj2)         # AdvancedClass(5)
print(obj1 * 3)            # AdvancedClass(30)
print(obj1 > obj2)         # True
print(len(obj1))           # 3
print(obj1[0])             # 1
obj1[0] = 10
print(obj1.data)           # [10, 2, 3]
print(2 in obj1)           # True
for item in obj1:
    print(item)             # 10, 2, 3
print(obj1(5))             # 50

with obj1 as ctx:
    print("Inside context")

```

2. Property Methods

python

```
class Employee:
    def __init__(self, name, salary, birth_year):
        self._name = name
        self._salary = salary
        self._birth_year = birth_year
        self._email = f"{name.lower()}@company.com"
        self._vacation_days = 20

    # Getter property
    @property
    def name(self):
        return self._name

    # Setter property
    @name.setter
    def name(self, value):
        if not value or not value.strip():
            raise ValueError("Name cannot be empty")
        old_name = self._name
        self._name = value.strip()
        # Update email when name changes
        self._email = f"{value.lower().replace(' ', '.')}@company.com"
        print(f"Name changed from {old_name} to {self._name}")

    # Deletable property
    @name.deleter
    def name(self):
        print(f"Deleting name {self._name}")
        del self._name

    @property
    def salary(self):
        return self._salary

    @salary.setter
    def salary(self, value):
        if value < 0:
            raise ValueError("Salary cannot be negative")
        if value < self._salary * 0.5:
            raise ValueError("Salary cannot be reduced by more than 50%")
        self._salary = value

    @property
    def birth_year(self):
        return self._birth_year
```



```

@property
def age(self):
    from datetime import datetime
    return datetime.now().year - self._birth_year

@property
def email(self):
    return self._email

@property
def vacation_days(self):
    return self._vacation_days

@vacation_days.setter
def vacation_days(self, days):
    if days < 0:
        raise ValueError("Vacation days cannot be negative")
    if days > 40:
        raise ValueError("Maximum 40 vacation days allowed")
    self._vacation_days = days

# Computed property
@property
def annual_salary(self):
    return self._salary * 12

@property
def pay_grade(self):
    if self._salary >= 100000:
        return "Executive"
    elif self._salary >= 75000:
        return "Senior"
    elif self._salary >= 50000:
        return "Mid"
    else:
        return "Junior"

# Usage
emp = Employee("John Doe", 60000, 1990)

# Getting properties
print(emp.name)           # John Doe
print(emp.email)          # john.doe@company.com
print(emp.age)            # 34 (if current year is 2024)
print(emp.salary)         # 60000
print(emp.annual_salary)  # 720000
print(emp.pay_grade)      # Mid

```

```

# Setting properties
emp.name = "Jane Smith"    # Name changed from John Doe to Jane Smith
print(emp.email)           # jane.smith@company.com
emp.salary = 70000
print(emp.salary)          # 70000

# Error cases
# emp.salary = -1000        # ValueError: Salary cannot be negative
# emp.salary = 20000        # ValueError: Salary cannot be reduced by more than 50%
# emp.vacation_days = 50    # ValueError: Maximum 40 vacation days allowed

# Deleting property
# del emp.name              # Deleting name Jane Smith

```

3. Classmethod vs Staticmethod vs Instancemethod

python

```

class MethodDemo:
    class_var = "Class variable"

    def __init__(self, value):
        self.instance_var = value

    # Instance method - uses self
    def instance_method(self, x):
        print(f"Instance: {self.instance_var}, x={x}")
        return self.instance_var + x

    # Class method - uses cls
    @classmethod
    def class_method(cls, x):
        print(f"Class: {cls.class_var}, x={x}")
        return cls.class_var + str(x)

    # Static method - no self or cls
    @staticmethod
    def static_method(x):
        print(f"Static: x={x}")
        return x * 2

# Usage
obj = MethodDemo(10)

# Instance method
print(obj.instance_method(5)) # 15

```

```
# MethodDemo.instance_method(obj, 5) # Can call with class

# Class method
print(MethodDemo.class_method(5)) # Class variable 5
print(obj.class_method(5)) # Can also call from instance

# Static method
print(MethodDemo.static_method(5)) # 10
print(obj.static_method(5)) # 10

# Differences
print(type(obj.instance_method)) # <class 'method'>
print(type(MethodDemo.class_method)) # <class 'method'>
print(type(MethodDemo.static_method)) # <class 'function'>
```

SPECIAL ATTRIBUTES & METHODS {#special}

1. slots (Memory Optimization)

python

```
class RegularClass:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class SlottedClass:
    __slots__ = ['name', 'age'] # Restricts attributes

    def __init__(self, name, age):
        self.name = name
        self.age = age

# Regular class - can add any attribute
reg = RegularClass("Alice", 20)
reg.phone = "123-456" # Works
reg.address = "NYC" # Works

# Slotted class - only allowed attributes
slot = SlottedClass("Bob", 25)
slot.age = 26 # Works
# slot.phone = "456-789" # ERROR! No attribute 'phone'

# Memory usage difference (in real applications)
```

```
import sys
reg_list = [RegularClass("Name", i) for i in range(1000)]
slot_list = [SlottedClass("Name", i) for i in range(1000)]

print(sys.getsizeof(reg_list[0])) # ~56 bytes
print(sys.getsizeof(slot_list[0])) # ~40 bytes (less memory)
```

2. new (Object Creation)

python

```
class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            print("Creating new instance")
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self, value):
        if not hasattr(self, 'initialized'):
            print("Initializing instance")
            self.value = value
            self.initialized = True

class ImmutablePoint:
    def __new__(cls, x, y):
        # Return existing object if values match
        if hasattr(cls, '_cache'):
            for obj in cls._cache:
                if obj.x == x and obj.y == y:
                    return obj
            else:
                cls._cache = []

        # Create new object
        instance = super().__new__(cls)
        cls._cache.append(instance)
        return instance

    def __init__(self, x, y):
        self.x = x
        self.y = y
```

Usage

```

s1 = Singleton("First")
s2 = Singleton("Second")
print(s1 is s2) # True (same instance)
print(s1.value) # First (initialized first time)
print(s2.value) # First (not re-initialized)

p1 = ImmutablePoint(1, 2)
p2 = ImmutablePoint(1, 2)
p3 = ImmutablePoint(3, 4)
print(p1 is p2) # True (cached)
print(p1 is p3) # False (different)

```

3. call (Making Objects Callable)

python

```

class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, x):
        return x * self.factor

class Counter:
    def __init__(self):
        self.count = 0

    def __call__(self):
        self.count += 1
        return self.count

class Logger:
    def __init__(self, prefix="LOG"):
        self.prefix = prefix

    def __call__(self, message, level="INFO"):
        import datetime
        timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        return f"{timestamp} [{self.prefix}:{level}] {message}"

# Usage
double = Multiplier(2)
triple = Multiplier(3)

print(double(10)) # 20
print(triple(10)) # 30

```

```

counter = Counter()
print(counter()) # 1
print(counter()) # 2
print(counter()) # 3

logger = Logger("APP")
print(logger("User logged in"))
print(logger("Database error", "ERROR"))

# Can be used as decorator
class Trace:
    def __call__(self, func):
        def wrapper(*args, **kwargs):
            print(f"Calling {func.__name__}")
            result = func(*args, **kwargs)
            print(f"{func.__name__} returned {result}")
            return result
        return wrapper

@Trace()
def add(a, b):
    return a + b

add(5, 3) # Logs: Calling add, add returned 8

```

4. Descriptors

python

```

class ValidatedAttribute:
    def __init__(self, min_value=None, max_value=None, required_type=None):
        self.min_value = min_value
        self.max_value = max_value
        self.required_type = required_type
        self.name = None

    def __set_name__(self, owner, name):
        self.name = name

    def __get__(self, instance, owner):
        if instance is None:
            return self
        return instance.__dict__.get(self.name)

    def __set__(self, instance, value):

```

```

# Type validation
if self.required_type and not isinstance(value, self.required_type):
    raise TypeError(f"{self.name} must be {self.required_type.__name__}")

# Range validation
if self.min_value is not None and value < self.min_value:
    raise ValueError(f"{self.name} must be >= {self.min_value}")
if self.max_value is not None and value > self.max_value:
    raise ValueError(f"{self.name} must be <= {self.max_value}")

instance.__dict__[self.name] = value

def __delete__(self, instance):
    del instance.__dict__[self.name]

class Person:
    name = ValidatedAttribute(required_type=str)
    age = ValidatedAttribute(min_value=0, max_value=150, required_type=int)
    height = ValidatedAttribute(min_value=50, max_value=250, required_type=float)
    salary = ValidatedAttribute(min_value=0, required_type=(int, float))

    def __init__(self, name, age, height, salary):
        self.name = name
        self.age = age
        self.height = height
        self.salary = salary

# Usage
p = Person("Alice", 30, 165.5, 50000)
print(p.name)    # Alice
print(p.age)     # 30
p.salary = 60000 # Works
# p.age = -5     # ValueError: age must be >= 0
# p.age = 200    # ValueError: age must be <= 150
# p.name = 123   # TypeError: name must be str

```

● COMPLETE REAL-WORLD EXAMPLE {#examples}

Library Management System with All Concepts

python

```

from abc import ABC, abstractmethod
from typing import List, Optional, Dict, Any

```

```

from datetime import datetime, timedelta
import uuid
import json

# ===== ABSTRACT CLASSES =====
class LibraryItem(ABC):
    """Abstract base class for all library items"""

    def __init__(self, title: str, author: str, item_id: str = None):
        self.title = title
        self.author = author
        self.item_id = item_id or str(uuid.uuid4())
        self._is_borrowed = False
        self._borrower = None
        self._borrow_date = None
        self._due_date = None
        self._reservation_list = []

    @abstractmethod
    def get_type(self) -> str:
        """Return type of library item"""
        pass

    @abstractmethod
    def get_loan_period(self) -> int:
        """Return loan period in days"""
        pass

    @property
    @abstractmethod
    def is_available(self) -> bool:
        """Check if item is available"""
        pass

    def borrow(self, user_id: str) -> bool:
        """Borrow item"""
        if not self.is_available:
            return False

        self._is_borrowed = True
        self._borrower = user_id
        self._borrow_date = datetime.now()
        self._due_date = self._borrow_date + timedelta(days=self.get_loan_period())
        return True

    def return_item(self) -> bool:
        """Return item"""
        if not self._is_borrowed:
            return False

```



```

        self._is_borrowed = False
        self._borrower = None
        self._borrow_date = None
        self._due_date = None
        return True

    def reserve(self, user_id: str) -> bool:
        """Reserve item"""
        if self.is_available:
            return False

        if user_id in self._reservation_list:
            return False

        self._reservation_list.append(user_id)
        return True

    def cancel_reservation(self, user_id: str) -> bool:
        """Cancel reservation"""
        if user_id in self._reservation_list:
            self._reservation_list.remove(user_id)
            return True
        return False

    @property
    def status(self) -> Dict[str, Any]:
        """Get item status"""
        return {
            "id": self.item_id,
            "title": self.title,
            "type": self.get_type(),
            "available": self.is_available,
            "borrower": self._borrower,
            "due_date": self._due_date.strftime("%Y-%m-%d") if self._due_date else None,
            "reservations": len(self._reservation_list)
        }

# ===== CONCRETE CLASSES =====
class Book(LibraryItem):
    """Book class"""

    def __init__(self, title: str, author: str, isbn: str, pages: int, item_id: str = None):
        super().__init__(title, author, item_id)
        self.isbn = isbn
        self.pages = pages
        self.__is_rare = False # Private attribute

```

```

def get_type(self) -> str:
    return "Book"

def get_loan_period(self) -> int:
    return 14 if not self.__is_rare else 7

@property
def is_available(self) -> bool:
    return not self._is_borrowed

def mark_rare(self):
    self.__is_rare = True

def __str__(self):
    return f"Book: {self.title} by {self.author} (ISBN: {self.isbn})"

class DVD(LibraryItem):
    """DVD class"""

    def __init__(self, title: str, director: str, duration: int, genre: str, item_id: str
= None):
        super().__init__(title, director, item_id)
        self.director = director
        self.duration = duration # minutes
        self.genre = genre

    def get_type(self) -> str:
        return "DVD"

    def get_loan_period(self) -> int:
        return 7

    @property
    def is_available(self) -> bool:
        return not self._is_borrowed

class Magazine(LibraryItem):
    """Magazine class"""

    def __init__(self, title: str, publisher: str, issue_number: int, year: int, item_id:
str = None):
        super().__init__(title, publisher, item_id)
        self.publisher = publisher
        self.issue_number = issue_number
        self.year = year

    def get_type(self) -> str:

```

```

        return "Magazine"

    def get_loan_period(self) -> int:
        return 7

    @property
    def is_available(self) -> bool:
        return not self._is_borrowed and len(self._reservation_list) == 0

# ===== USER CLASS =====
class User:
    """User class with class attributes and methods"""

    max_borrow_limit = 5
    total_users = 0

    def __init__(self, name: str, email: str, phone: str):
        self.user_id = str(uuid.uuid4())
        self.name = name
        self._email = email # Protected
        self.__phone = phone # Private
        self.borrowed_items = []
        self.reserved_items = []
        self.fines = 0.0
        User.total_users += 1

    @property
    def email(self):
        return self._email

    @email.setter
    def email(self, value):
        if "@" not in value:
            raise ValueError("Invalid email format")
        self._email = value

    @property
    def phone(self):
        return self.__phone

    @phone.setter
    def phone(self, value):
        if len(value) < 10:
            raise ValueError("Phone number too short")
        self.__phone = value

    def borrow_item(self, item: LibraryItem) -> bool:
        if len(self.borrowed_items) >= self.max_borrow_limit:

```

```

        return False

    if item.borrow(self.user_id):
        self.borrowed_items.append(item.item_id)
        return True
    return False

def return_item(self, item: LibraryItem) -> bool:
    if item.return_item():
        if item.item_id in self.borrowed_items:
            self.borrowed_items.remove(item.item_id)

        # Check for late returns
        if item._due_date and item._due_date < datetime.now():
            days_late = (datetime.now() - item._due_date).days
            self.fines += days_late * 1.0 # $1 per day

        return True
    return False

def reserve_item(self, item: LibraryItem) -> bool:
    if item.reserve(self.user_id):
        self.reserved_items.append(item.item_id)
        return True
    return False

def get_info(self) -> Dict[str, Any]:
    return {
        "id": self.user_id,
        "name": self.name,
        "email": self._email,
        "borrowed": len(self.borrowed_items),
        "fines": self.fines
    }

    @classmethod
    def create_from_dict(cls, data: Dict[str, str]):
        """Factory method to create user from dict"""
        return cls(data["name"], data["email"], data["phone"])

    @staticmethod
    def is_valid_email(email: str) -> bool:
        """Validate email format"""
        return "@" in email and "." in email

# ===== LIBRARY CLASS =====
class Library:
    """Main library class with composition"""

```

```

def __init__(self, name: str, address: str):
    self.name = name
    self.address = address
    self._inventory = {} # item_id -> LibraryItem
    self._users = {}      # user_id -> User
    self._transactions = []
    self.__opening_hours = "9 AM - 9 PM" # Private

# Property for opening hours
@property
def opening_hours(self):
    return self.__opening_hours

@opening_hours.setter
def opening_hours(self, hours):
    if not hours or "AM" not in hours:
        raise ValueError("Invalid opening hours format")
    self.__opening_hours = hours

def add_item(self, item: LibraryItem) -> None:
    self._inventory[item.item_id] = item

def remove_item(self, item_id: str) -> bool:
    if item_id in self._inventory:
        del self._inventory[item_id]
        return True
    return False

def register_user(self, user: User) -> None:
    self._users[user.user_id] = user

def find_item(self, title: str) -> Optional[LibraryItem]:
    for item in self._inventory.values():
        if item.title.lower() == title.lower():
            return item
    return None

def find_items_by_author(self, author: str) -> List[LibraryItem]:
    return [item for item in self._inventory.values()
            if hasattr(item, 'author') and item.author.lower() == author.lower()]

def borrow_item(self, user_id: str, item_id: str) -> bool:
    if user_id not in self._users or item_id not in self._inventory:
        return False

    user = self._users[user_id]
    item = self._inventory[item_id]

```

```

    if user.borrow_item(item):
        self._transactions.append({
            "type": "borrow",
            "user": user_id,
            "item": item_id,
            "date": datetime.now().isoformat()
        })
        return True
    return False

def return_item(self, user_id: str, item_id: str) -> bool:
    if user_id not in self._users or item_id not in self._inventory:
        return False

    user = self._users[user_id]
    item = self._inventory[item_id]

    if user.return_item(item):
        self._transactions.append({
            "type": "return",
            "user": user_id,
            "item": item_id,
            "date": datetime.now().isoformat()
        })
        return True
    return False

def get_inventory_status(self) -> Dict[str, Any]:
    total = len(self._inventory)
    available = sum(1 for item in self._inventory.values() if item.is_available)
    borrowed = total - available

    return {
        "total": total,
        "available": available,
        "borrowed": borrowed,
        "items": [item.status for item in self._inventory.values()]
    }

def get_user_status(self, user_id: str) -> Optional[Dict[str, Any]]:
    if user_id not in self._users:
        return None

    user = self._users[user_id]
    info = user.get_info()
    info["borrowed_items"] = [
        self._inventory[item_id].title

```

```

        for item_id in user.borrowed_items
        if item_id in self._inventory
    ]
    return info

# Static method - utility
@staticmethod
def calculate_fine(user: User) -> float:
    return user.fines

# Class method - Library statistics
@classmethod
def get_library_stats(cls, libraries: List['Library']) -> Dict[str, int]:
    total_items = sum(len(lib._inventory) for lib in libraries)
    total_users = sum(len(lib._users) for lib in libraries)
    return {
        "total_libraries": len(libraries),
        "total_items": total_items,
        "total_users": total_users
    }

# Magic methods
def __str__(self):
    return f"Library: {self.name} ({len(self._inventory)} items)"

def __len__(self):
    return len(self._inventory)

def __getitem__(self, item_id):
    return self._inventory.get(item_id)

def __contains__(self, item_id):
    return item_id in self._inventory

def __iter__(self):
    return iter(self._inventory.values())

# ===== DEMONSTRATION =====
def main():
    # Create Library
    library = Library("City Library", "123 Main St")
    print(library.opening_hours) # 9 AM - 9 PM

    # Create items
    book1 = Book("Python Programming", "John Doe", "978-1234567890", 500)
    book2 = Book("Data Science", "Jane Smith", "978-0987654321", 400)
    dvd1 = DVD("Inception", "Christopher Nolan", 148, "Sci-Fi")
    mag1 = Magazine("National Geographic", "NatGeo", 202, 2024)

```

```
# Add items to library
library.add_item(book1)
library.add_item(book2)
library.add_item(dvd1)
library.add_item(mag1)

# Create users
user1 = User("Alice Johnson", "alice@email.com", "1234567890")
user2 = User("Bob Smith", "bob@email.com", "9876543210")

# Register users
library.register_user(user1)
library.register_user(user2)

# Borrow items
print("\n=== Borrowing ===")
print(f"Borrowing book: {library.borrow_item(user1.user_id, book1.item_id)}")
print(f"Book status: {book1.is_available}")

# Try to borrow same book
print(f"Try borrow same book: {library.borrow_item(user2.user_id, book1.item_id)}")

# Return item
print("\n=== Returning ===")
print(f"Returning book: {library.return_item(user1.user_id, book1.item_id)}")
print(f"Book status: {book1.is_available}")

# Get inventory status
print("\n=== Inventory Status ===")
status = library.get_inventory_status()
print(f"Total items: {status['total']}")
print(f"Available: {status['available']}")
print(f"Borrowed: {status['borrowed']}")

# Get user status
print("\n=== User Status ===")
user_status = library.get_user_status(user1.user_id)
print(f"User: {user_status['name']}")
print(f"Borrowed items: {user_status['borrowed_items']}")
print(f"Fines: ${user_status['fines']}")

# Using magic methods
print("\n=== Magic Methods ===")
print(str(library)) # __str__
print(len(library)) # __len__
print(book1.item_id in library) # __contains__
```



```
# Iterate items
for item in library:
    print(f" - {item.title}")

# Class method usage
print("\n=== Class Methods ===")
stats = Library.get_library_stats([library])
print(stats)

# Static method
print(f"Valid email: {User.is_valid_email('test@email.com')}")

# Property example
try:
    library.opening_hours = "invalid"
except ValueError as e:
    print(f"Error: {e}")

# Abstract class implementation
print("\n=== Abstract Methods ===")
print(f"Book loan period: {book1.get_loan_period()} days")
print(f"DVD loan period: {dvd1.get_loan_period()} days")

# Composition
print("\n=== Composition ===")
print(f"Library has {len(library)} items")
print(f"Library has {len(library._users)} users")

# Run demonstration
if __name__ == "__main__":
    main()
```



COMPLETE REFERENCE TABLE

Attribute Types

Type	Syntax	Scope	Access
Instance	<code>self.var</code>	Per object	Inside/outside
Class	<code>Class.var</code>	All objects	Inside/outside
Private	<code>self.__var</code>	Class only	Inside only

Type	Syntax	Scope	Access
Protected	<code>self._var</code>	Class/subclass	Convention
Property	<code>@property</code>	Per object	Controlled

Method Types

Type	Decorator	First Param	Call	Usage
Instance	None	<code>self</code>	<code>obj.method()</code>	Object operations
Class	<code>@classmethod</code>	<code>cls</code>	<code>Class.method()</code>	Factory/alt construc
Static	<code>@staticmethod</code>	None	<code>Class.method()</code>	Utility functions
Abstract	<code>@abstractmethod</code>	None	Must implement	Contract definition
Property	<code>@property</code>	None	<code>obj.prop</code>	Controlled attributes
Magic	<code>__method__</code>	Various	Special syntax	Operator overloadin

Abstract Class Components

Component	Syntax	Required	Purpose
Abstract class	<code>class Class(ABC):</code>	Yes	Cannot instantiate
Abstract method	<code>@abstractmethod def method():</code>	Yes	Must implement
Abstract property	<code>@property @abstractmethod</code>	Yes	Must implement
Concrete method	<code>def method():</code>	No	Can use as-is
Concrete property	<code>@property def prop():</code>	No	Can use as-is

 SUMMARY & BEST PRACTICES

When to Use What

1. **Instance Methods:** When you need to access/modify instance attributes
2. **Class Methods:** When you need class-level operations or factory methods
3. **Static Methods:** When function is logically related to class but doesn't need class/instance
4. **Abstract Methods:** When you want to enforce implementation in subclasses
5. **Properties:** When you want controlled access to attributes
6. **Magic Methods:** When you want custom behavior for operators or built-in functions

Best Practices

- ✓ **Always use** `self` for instance methods
- ✓ **Use** `cls` for class methods
- ✓ **Use** `@abstractmethod` for required methods in abstract classes
- ✓ **Use properties** instead of direct attribute access
- ✓ **Use private attributes** for internal state
- ✓ **Use class methods** as alternative constructors
- ✓ **Use static methods** for utility functions
- ✓ **Document** abstract methods with clear docstrings
- ✓ **Keep abstraction levels** consistent
- ✓ **Don't overuse** abstract classes - use when needed

🎉 **Congratulations!** You now have complete understanding of OOP methods, attributes, and abstract concepts in Python!